

Design Smells Analysis

Design Smell 1: Broken Hierarchy in WeblogPermission

Category: Broken Hierarchy

Description

The WeblogPermission class exhibits Broken Hierarchy, a design smell where inheritance relationships are misused or improperly structured, leading to violated expectations of the inheritance contract. Specifically, WeblogPermission extends ObjectPermission which extends RollerPermission which extends java.security.Permission, creating a deep inheritance chain (DIT=2) where the subclass doesn't properly fulfill the contract of its superclasses.

The hierarchy violation manifests in several ways:

1. **Inconsistent equals() implementation:** WeblogPermission overrides equals() with a "Long Statement" implementation that's complex and potentially violates the equals contract
2. **Cyclic dependencies:** WeblogPermission participates in cyclic dependencies, suggesting architectural confusion
3. **High LCOM (1.0):** Perfect lack of cohesion means methods don't work together, indicating the class tries to do too much independently rather than leveraging inheritance properly

Supporting Evidence

1. Designite Analysis - Design Code Smells

From designCodeSmells.csv:

```
.,org.apache.roller.weblogger.pojos,WeblogPermission,Deficient Encapsulation
.,org.apache.roller.weblogger.pojos,WeblogPermission,Broken Hierarchy
.,org.apache.roller.weblogger.pojos,WeblogPermission,Cyclic-Dependent Modularization
```

WeblogPermission is explicitly flagged for "Broken Hierarchy" along with two other serious design smells.

2. Type Metrics Analysis

From typeMetrics.csv:

WeblogPermission: NOF=4, NOPF=4, NOM=10, NOPM=10, LOC=92, WMC=24, DIT=2, LCOM=1.0, FANIN=27, FANOUT=4

ObjectPermission: NOF=7, NOM=14, NOPM=14, LOC=54, WMC=14, DIT=1, LCOM=0.571
RollerPermission: NOF=1, NOM=11, NOPM=11, LOC=72, WMC=18, DIT=0, LCOM=0.182

Key Indicators:

- **DIT=2:** Two-level inheritance depth from RollerPermission through ObjectPermission
- **LCOM=1.0:** Perfect lack of cohesion (worst possible value), methods are completely independent
- **High FANIN=27:** 27 classes depend on this flawed hierarchy
- **High WMC=24:** Complex implementation suggests inheritance not simplifying design
- **All methods public (NOPM=10 out of NOM=10):** Suggests encapsulation issues compounding hierarchy problems

3. Implementation Code Smells

From implementationCodeSmells.csv:

```
.,org.apache.roller.weblogger.pojos,WeblogPermission,implies,Complex Method  
.,org.apache.roller.weblogger.pojos,WeblogPermission,equals,Long Statement
```

Critical Evidence:

- **Complex implies() method:** Permission checking logic is overly complex
- **Long Statement in equals():** Suggests the equals contract is difficult to implement correctly in this hierarchy

4. SonarQube Analysis Evidence

ObjectPermission.java issues:

- 3 open maintainability issues with "Medium" severity
- Constructor visibility and design issues
- RollerPermission.java shows 1 maintainability issue with "Medium" severity
- WeblogPermission.java shows 2 maintainability issues

5. Evidence from UML Class Diagram

From the UML class diagram, the permission hierarchy exposes multiple structural inconsistencies that collectively indicate a broken and poorly abstracted inheritance design:

- **Asymmetric inheritance chain:**
GlobalPermission extends RollerPermission directly, while WeblogPermission extends ObjectPermission, making WeblogPermission the only class that traverses the full three-level hierarchy.
- **Non-abstract intermediate base class:**
ObjectPermission has a public constructor and is not marked abstract, yet no class instantiates it directly. Its sole role is to sit between RollerPermission and WeblogPermission, which is the semantic role of an abstract class.

- **Shadowed state in subclasses:**
GlobalPermission redeclares the actions field and its getActions() / setActions() methods even though these are already defined in RollerPermission, creating two independent action-storage paths within the same hierarchy.
- **Constructor breaks superclass contract:**
WeblogPermission provides a constructor that omits the User parameter, allowing the inherited userName field from ObjectPermission to remain uninitialised, violating the superclass's implied invariants.
- **Inverted dependency in root class:**
RollerPermission declares addAction(permission : ObjectPermission), coupling the root of the hierarchy to one of its own descendants and reversing the intended abstraction direction.
- **Inconsistent encapsulation policy:**
ObjectPermission protects its state using protected fields, while WeblogPermission exposes permission constants as public fields, leaving the hierarchy with no consistent encapsulation boundary.

Impact Analysis

Consequences of Broken Hierarchy:

1. **Violated Liskov Substitution Principle:** Subclasses cannot be safely substituted for superclasses
2. **Complex Permission Logic:** implies() method complexity (flagged by Designite) indicates inheritance isn't simplifying design
3. **High Maintenance Risk:** 27 classes depend on this flawed hierarchy (FANIN=27)
4. **Testing Difficulty:** LCOM=1.0 means methods can't be tested cohesively
5. **Equals/HashCode Errors:** Long Statement in equals() increases bug risk in collections
6. **Cyclic Dependencies:** Broken hierarchy participates in architectural cycles

Design Smell 2: Cyclic-Dependent Modularization

Category: Cyclic-Dependent Modularization

Description

Cyclic-Dependent Modularization occurs when two or more modules depend on each other either directly or transitively, violating the Acyclic Dependency Principle. Such cycles break layered architecture assumptions and make the system fragile, tightly coupled, and difficult to evolve.

The core issue is that domain-layer POJOs, especially User and GlobalPermission, contain business and UI logic, causing upward dependencies into the Business and UI layers.

- User (a data-layer POJO) directly calls:
 - WebloggerFactory.getWeblogger().getUserManager() (Business layer)

- `RollerContext.getPasswordEncoder()` (UI layer)
- `GlobalPermission` (also a POJO) queries role data via `UserManager`
- `UserManager` operates on `User` and permission objects, closing the dependency loop

This results in multiple interlocking dependency cycles across data, business, UI, and infrastructure layers, violating Roller's intended 3-tier architecture.

Supporting Evidence

1. Designite Analysis – Design Code Smells

From `designCodeSmells.csv`, Designite explicitly flags multiple core classes for Cyclic-Dependent Modularization:

```
org.apache.roller.weblogger.pojos.User
org.apache.roller.weblogger.pojos.GlobalPermission
org.apache.roller.weblogger.ui.core.RollerContext
```

2. Type Metrics Analysis – FANIN / FANOUT Patterns

Coupling metrics extracted from `typeMetrics.csv` show the following patterns:

- User: FANIN = 121, FANOUT = 7
Interpretation: A domain POJO should ideally have zero outgoing dependencies. FANOUT = 7 indicates inappropriate dependencies on business and UI layers.
- GlobalPermission: FANIN = 17, FANOUT = 6
Interpretation: High bidirectional coupling, indicating participation in dependency cycles.
- UserManager: FANIN = 34, FANOUT = 3
Interpretation: Central business component with dependencies in both directions.
- RollerContext: FANIN = 28, FANOUT = 5
Interpretation: UI-layer utility entangled in cyclic dependencies.

Key indicators:

- POJO classes exhibit non-zero FANOUT values.
- Multiple classes have both high FANIN and FANOUT, a classic sign of cyclic-dependent modularization.
- Bidirectional dependencies exist across architectural layers.

3. Implementation Evidence – Empty Catch Clauses

From `implementationCodeSmells.csv`:

```
User.hasGlobalPermissions – Empty catch clause
JPAUserManagerImpl.checkPermission – Empty catch clause
JPAUserManagerImpl.grantWeblogPermission – Empty catch clause
```

- Empty catches often indicate error-propagation problems
- Common symptom when exceptions cannot cross cyclic boundaries cleanly

4. Evidence from UML Class Diagram

From the UML analysis:

Cycle A – User ↔ GlobalPermission ↔ UserManager

- User.hasGlobalPermissions()
- GlobalPermission(User)
- UserManager.checkPermission(permission, user)

Cycle B – User ↔ RollerContext

- User.resetPassword()
- RollerContext.getPasswordEncoder()
- UI session objects store User

Hidden Dependency

- Weblog stores creator as String
- Exposes creator as User
- Implies hidden runtime dependency on user resolution

All cycles are statically visible via method signatures and associations.

Impact Analysis

Consequences of Cyclic-Dependent Modularization

- **Compilation Issues:** Modules cannot be compiled independently, the entire subsystem must be built together.
- **Testing Difficulty:** POJOs cannot be unit tested alone and require both business and UI layers to be set up.
- **Change Amplification:** Minor changes spread across multiple layers, leading to deprecated code and abandoned refactorings.
- **High Cognitive Load:** Developers must understand large portions of the system to modify even small methods.
- **Slower Build Times:** Cyclic dependencies prevent parallel builds, increasing build time.
- **Refactoring Risk:** Breaking cycles requires coordinated changes across many classes, increasing regression risk.
- **Architectural Violations:** Violates the Dependency Inversion Principle, Acyclic Dependency Principle, and layered architecture rules.